BSc (Hons) in **Information Technology Specialized in AI**

Faculty of Computing

IT3012 : Intelligent Agents

Year 3 · Semester 1 · 2026 Semester 2

Practical 01

> **Objective & Lecture Mapping:** Recall that an intelligent agent acts within a continuous loop: Sensors generate percept sequences, and Actuators execute actions to maximize a Performance measure. Use this sheet to execute practical modifications and incrementally evaluate your design against Lecture 01 and 02 theory (from basic recall to analytical classification).

Part 1: Code Analysis & PEAS Evaluation

Task 0 : Setting up and getting the starter Code

1. Go to the Git Hub Repo. Create a Fork of this Git Repo to your Personal Github Profile. Open it using your favourite IDE and follow the below instruction to complete the practical. The base code is in the "master" brach of the repo.

Task 1.1: The Environment State (__init__)

1. (Remember) List the four components of the PEAS framework discussed in Lecture 01.

Performance measure, Environment, Actuators, and Sensors.

2. (Understand) Look at the `VisualGridHuntGame.__init__` method. Which specific variables in this code represent the physical "Environment (E)" state?

self.width, self.height, self.walls, and self.food_positions define the physical Environment (E). self.opponents also belongs to E because it represents other entities that occupy and change the world state.

3. (Analyze) Based on the variables initialized (specifically `self.opponents`), classify this baseline environment as "Single-Agent" or "Multi-Agent". Briefly justify your choice.

This is a Multi-Agent environment. self.opponents is a non-empty list of independently moving entities (default num_opponents=2) that act each turn and can collide with the agent, directly affecting its score. Therefore, it is multi-agent rather than single-agent.

Task 1.2: The Perception Subsystem (get_percept)

4. (Remember) Define what a "percept sequence" is according to Lecture 01.

A percept sequence is the complete history of every percept an agent has received from its sensors up to the current point in time. The agent's action at any moment can, in principle, depend on this entire sequence, not just the current percept.

5. (Understand) Which component of the PEAS framework does the `get_percept()` method represent in our code?

get_percept() implements the Sensors (S) component of PEAS. It is the mechanism through which the environment's state is converted into percept data that the agent receives.

6. (Evaluate) Based on the exact dictionary returned by `get_percept()`, is this environment "Fully Observable" or "Partially Observable"? Explain why based on what the agent can and cannot see.

opponent positions. It does not reveal the complete layout of food_positions or walls elsewhere on the grid. Therefore, the agent receives only partial information about the environment.

Task 1.3: Action Execution (execute_action)

7. (Understand) When `execute_action()` deducts points for hitting a wall, which PEAS component is being actively updated?

The Performance measure (P). self.score is the numeric performance metric, and deducting points for hitting a wall updates this performance measure.

8. (Evaluate) Why is it structurally important that this metric evaluates changes in the external environment state (hitting a wall) rather than the agent's internal processing effort?

judged by the results an agent achieves. If internal effort were rewarded instead of useful outcomes, an agent could receive a high score simply by performing extensive computation without accomplishing the task.

Part 2: Guided Modification & Deep Theory Mapping

> Follow the implementation instructions to inject a new hazard, then answer the questions to map your code changes back to the theoretical nature of the environment.

Step 2.1: Extending the Environment Initialization (__init__)

1. **Implementation Instructions:** Declare a new trap collection attribute (e.g., `self.toxic_traps = set()`) inside `__init__`. Populate it with coordinate tuples safely avoiding position `(0, 0)`, walls, and food.

9. (Understand) If you successfully add `self.toxic_traps` to the environment but intentionally **hide this data from the agent's sensors, how does this specifically alter the environment's "Observability" classification?**

environment now contains information about hazards that the agent cannot perceive, so the agent must make decisions using incomplete information and may be penalized by hazards it cannot directly detect.

Step 2.2: Updating the Perception Subsystem (get_percept)

1. **Implementation Instructions:** Locate `get_percept(self)` and add a new boolean sensor key (e.g., `'smells_toxin': tuple(self.agent_pos) in self.toxic_traps`) to the dictionary returned to the agent loop.

10. (Analyze) By adding `'smells_toxin'`, you expand the agent's percept sequence. Explain how this specific sensor helps the agent act with "Rationality" (maximizing expected utility).

about a toxic trap and can consider the expected cost of stepping onto it, such as the -15 score penalty. This allows the agent to make more informed decisions and choose actions that are more likely to maximize its expected utility.

Step 2.3: Modifying Action Execution & Visual Rendering (execute_action & draw_grid)

1. **Implementation Instructions:** In `execute_action`, check if the updated `self.agent_pos` intersects with `self.toxic_traps` to decrement `self.score` by 15 points. In `draw_grid`, iterate through traps to render custom purple shapes on the canvas.

11. (Remember) You programmed a severe score penalty for stepping on a trap to avoid metric exploitation. What was the classic "Vacuum World Exploit" example discussed in Lecture 01 that illustrated the danger of faulty metrics?

it again, allowing it to clean the same dirt repeatedly and receive rewards indefinitely. This shows that a performance measure must represent the actual desired outcome rather than simply rewarding the frequency of an action.

Finalize and Lock Lab 01 Analytical Evaluation

Lab 01 code analysis and theoretical evaluation complete and verified!